Fetching

2026. February 24., Tuesday 17:40

The four functions

Relative

- `LAG(column, n)` returns `column`'s value at the row `n` rows before the current row
- `LEAD(column, n)` returns `column`'s value at the row `n` rows after the current row

Absolute

- `FIRST_VALUE(column)` returns the first value in the table or partition
- `LAST_VALUE(column)` returns the last value in the table or partition

FIRST_VALUE and LAST_VALUE

Query

```sql
SELECT
  Year, City,
  FIRST_VALUE(City) OVER
    (ORDER BY Year ASC) AS First_City,
  LAST_VALUE(City) OVER (
   ORDER BY Year ASC
   RANGE BETWEEN
     UNBOUNDED PRECEDING AND
     UNBOUNDED FOLLOWING
  ) AS Last_City
FROM Hosts
ORDER BY Year ASC;
```

Result

```
| Year | City      | First_City | Last_City       |
|------|-----------|------------|-----------------|
| 1896 | Athens    | Athens     | London          |
| 1900 | Paris     | Athens     | London          |
| 1904 | St Louis  | Athens     | London          |
| 1908 | London    | Athens     | London          |
| 1912 | Stockholm | Athens     | London          |
```

- By default, a window starts at the beginning of the table or partition and ends at the current row
- `RANGE BETWEEN ...` clause extends the window to the end of the table or partition

Partitioning works with these four functions

The ranking functions

- `ROW_NUMBER()` always assigns unique numbers, even if two rows' values are the same
- `RANK()` assigns the same number to rows with identical values, skipping over the next numbers in such cases
- `DENSE_RANK()` also assigns the same number to rows with identical values, but doesn't skip over the next numbers

ROW_NUMBER() just assigns values
RANK() if there are two second, then 1,2,2,4...
DENSE_RANK() if there are two second, then 1,2,2,3...
Partitioning works with these functions as well

Paging

What is paging?

- **Paging:** Splitting data into (approximately) equal chunks
- **Uses**
  - Many APIs return data in "pages" to reduce data being sent
  - Separating data into quartiles or thirds (top middle 33%, and bottom thirds) to judge

performance

Enter NTILE

- `NTILE(n)` splits the data into `n` approximately equal pages

Paging

Query

```
WITH Disciplines AS (
  SELECT
    DISTINCT Discipline
  FROM Summer_Medals)

SELECT
  Discipline, NTILE(15) OVER () AS Page
From Disciplines
ORDER BY Page ASC;
```

Result

```
| Discipline          | Page |
|---------------------|------|
| Wrestling Freestyle | 1    |
| Archery             | 1    |
| Baseball            | 1    |
| Lacrosse            | 1    |
| Judo                | 1    |
| Athletics           | 2    |
| ...                 | ...  |
```